23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : B16
Team Members :

Roll No	Student Name
AM.SC.U4CSE25121	Harinandan Sunil
AM.SC.U4CSE25128	Kesav Hari
AM.SC.U4CSE25136	Nandagopal A
AM.SC.U4CSE25137	Navajyoth P Krishna

Project Title: Group Travel Itinerary and Automated Expense Settlement System

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	3-5
3	Requirements (Functional & Non-functional)	5-6
4	UI Design	7-11
5	Database Design	12-15
6	Database Selection & Justification	16-19
7	Implementation	20-24
8	Testing & Validation	25-28
9	Results & Screenshots	28-32
10	Challenges faced & solutions	32-33
11	Conclusion & future enhancements	33
	References	33

1. Abstract

Organizing group travel frequently causes financial confusion due to manual, decentralized expense tracking. The objective of this project is to build a "Group Travel Itinerary and Automated Expense Settlement System" to eliminate these disputes. The intended users are group travelers and trip coordinators who need a centralized, transparent ledger for shared schedules and payments. The proposed PostgreSQL database system securely manages user accounts, trip details, and financial transactions. Major functionalities include an interactive daily itinerary planner, a dynamic expense-splitting engine, and a debt-simplification calculator. Users can log shared bills with exact split ratios and view real-time outstanding balances. This system implements stored procedures to calculate optimized debt settlement paths, minimizing the number of peer-to-peer transactions required. Database triggers automatically update running balances, and complex SQL constraints guarantee that individual expense shares strictly match the total bill, preventing financial data anomalies.

2. Introduction

2.1 Background and Context

Group travel is fundamentally a collaborative endeavor, requiring the synchronization of multiple individuals across diverse logistical and financial dimensions. While the core objective of group travel is shared experience, the underlying mechanics involve complex coordination of transportation, accommodation, daily itineraries, and, most critically, shared expenses. As groups navigate through varied activities, financial responsibilities are inevitably distributed. One individual may pay for a hotel booking, another for a shared meal, and a third for local transit.

In traditional scenarios, tracking these decentralized payments relies heavily on informal methods. Travelers often attempt to maintain a mental ledger, collect paper receipts, or utilize basic digital tools such as shared spreadsheet applications or generic messaging groups. However, as the duration of the trip extends and the number of transactions increases, these informal methodologies rapidly degrade, leading to missing data, transactional disputes, and significant friction among group members.

2.2 Problem Statement

The primary challenge in managing group travel arises from the disconnect between event scheduling and financial accounting. Existing solutions typically address only one facet of the problem; dedicated itinerary planners lack robust financial ledgers, while dedicated expense-splitting applications operate in a vacuum, disconnected from the contextual timeline of the trip.

Furthermore, manual expense tracking is highly susceptible to data anomalies. When a single bill is split among multiple participants in varying ratios—such as unequal shares for a meal or exact currency amounts for a specific ticket—human error frequently results in unbalanced ledgers where the sum of individual debts does not equal the original bill. At the conclusion of a trip, groups face the daunting task of manual debt reconciliation. Calculating the "who owes whom" matrix across a high volume of cross-payments often results in a convoluted web of peer-to-peer transfers, forcing members to execute multiple, redundant bank transactions to settle their balances.

2.3 Proposed System Overview

To resolve these inherent inefficiencies, this project proposes **TripLedger**, a comprehensive, centralized relational database application designed to unify travel itineraries with an automated expense settlement system. TripLedger serves as a single source of truth for both group coordinators and standard members.

The application allows users to register, create customized travel environments (Trips), establish base currencies, and invite peers. Once a trip environment is initialized, users interact with a dual-pane system. The itinerary module tracks daily chronological activities, ensuring all members have real-time access to scheduling. The financial module acts as a strict, automated ledger where members log shared bills. When an expense is recorded, the system requires the user to specify exact split parameters (Equal, Exact, or Percentage) for involved members, explicitly tying the financial record to the corresponding itinerary event for complete contextual transparency.

2.4 Technical Motivation and Database Approach

Because financial data demands absolute accuracy and structural integrity, a robust Database Management System (DBMS) is the critical foundation of this project. PostgreSQL was selected as the underlying architecture due to its strict adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties and its advanced support for programmatic database logic.

The system shifts the burden of mathematical validation and data consistency from the front-end application directly to the database layer. Complex SQL constraints are implemented to guarantee that financial anomalies—such as an expense split failing to sum to the total bill—are automatically rejected before they can corrupt the ledger. Furthermore, the system heavily utilizes PostgreSQL’s event-driven architecture. Database triggers are deployed to automatically recalculate and update a user's net standing the exact millisecond a new expense is logged, modified, or deleted.

To address the end-of-trip reconciliation problem, the database integrates a custom stored procedure (simplify_trip_debts) written in pgSQL. This algorithmic routine evaluates the net balances of all users simultaneously and generates an optimized

settlement path. By matching the highest debtors with the highest creditors, the database minimizes the total number of peer-to-peer transactions required to achieve a zero-balance state across the group.

2.5 Scope and Objectives

The development and implementation of this project are guided by the following core objectives:

- **Centralized Relational Architecture:** Design and deploy an optimized, normalized PostgreSQL database schema that eliminates data redundancy and enforces referential integrity across users, trips, schedules, and expenses.
- **Contextual Financial Logging:** Provide a platform where shared expenses are permanently linked to the specific chronological events that generated them.
- **Absolute Data Integrity:** Implement robust backend constraints that strictly enforce mathematical rules on all financial entries, rendering unbalanced ledgers impossible.
- **Automated Real-Time Accounting:** Utilize database triggers to maintain an always-accurate, real-time snapshot of every member's net balance without requiring manual calculation.
- **Algorithmic Debt Resolution:** Engineer an automated simplification routine that digests complex group debts and outputs the minimum possible number of settlement transactions.
- **Responsive User Experience:** Deliver a clean, dynamic, and intuitive web-based interface that allows users to interact seamlessly with the underlying database through a secure Node.js API bridge.

3. Requirements

Functional Requirements

- **User Registration & Authentication:** New users must register using their full name, email address, phone number, and password, enabling secure login access.
- **Trip & Member Management:** Users can create trips (defining name, description, duration, currency, and coordinator ID) and manage member roles (Coordinator or Member).
- **Itinerary Scheduling:** Users can log and view daily chronological activities containing locations, activity types, days, timestamps, and notes.

- **Shared Expense Logging:** Members can record bills specifying amounts, currencies, categories, payer IDs, dates, and optional itinerary links.
- **Expense Splitting & Validation:** Expenses are divided via defined rules (Equal, Exact, Percentage), with the system validating that the sum of split amounts equals the total expense prior to transaction commit.
- **Automated Net Balance Calculation:** Each member's net standing is automatically updated via database triggers upon expense creation, modification, or deletion.
- **Debt Simplification (simplify_trip_debts):** A stored routine evaluates trip balances to generate an optimized transfer table, minimizing settlement transactions.

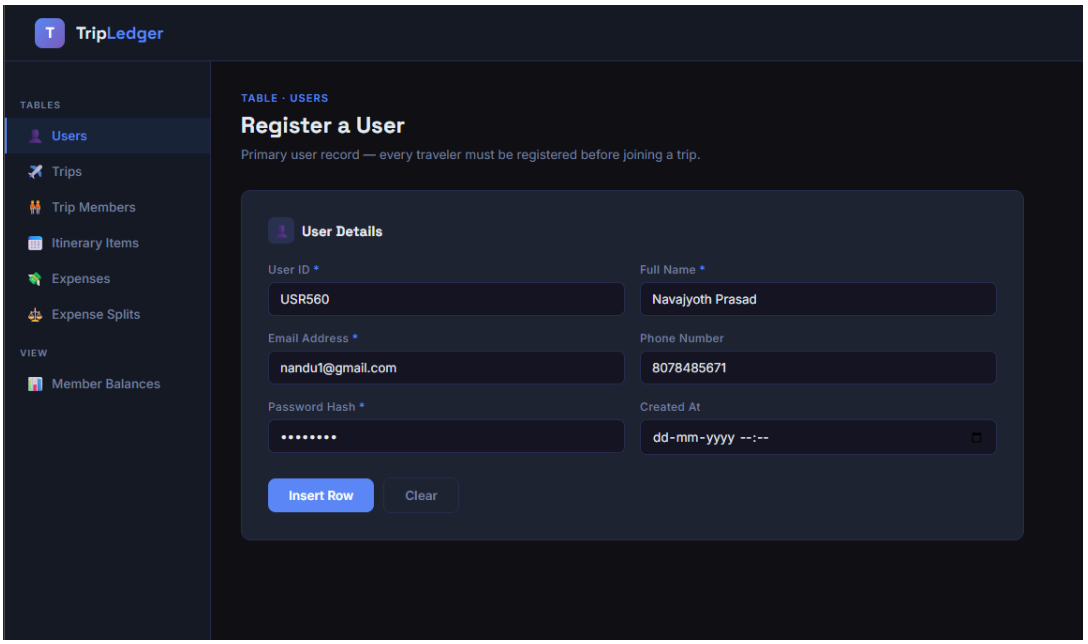
Non-Functional Requirements

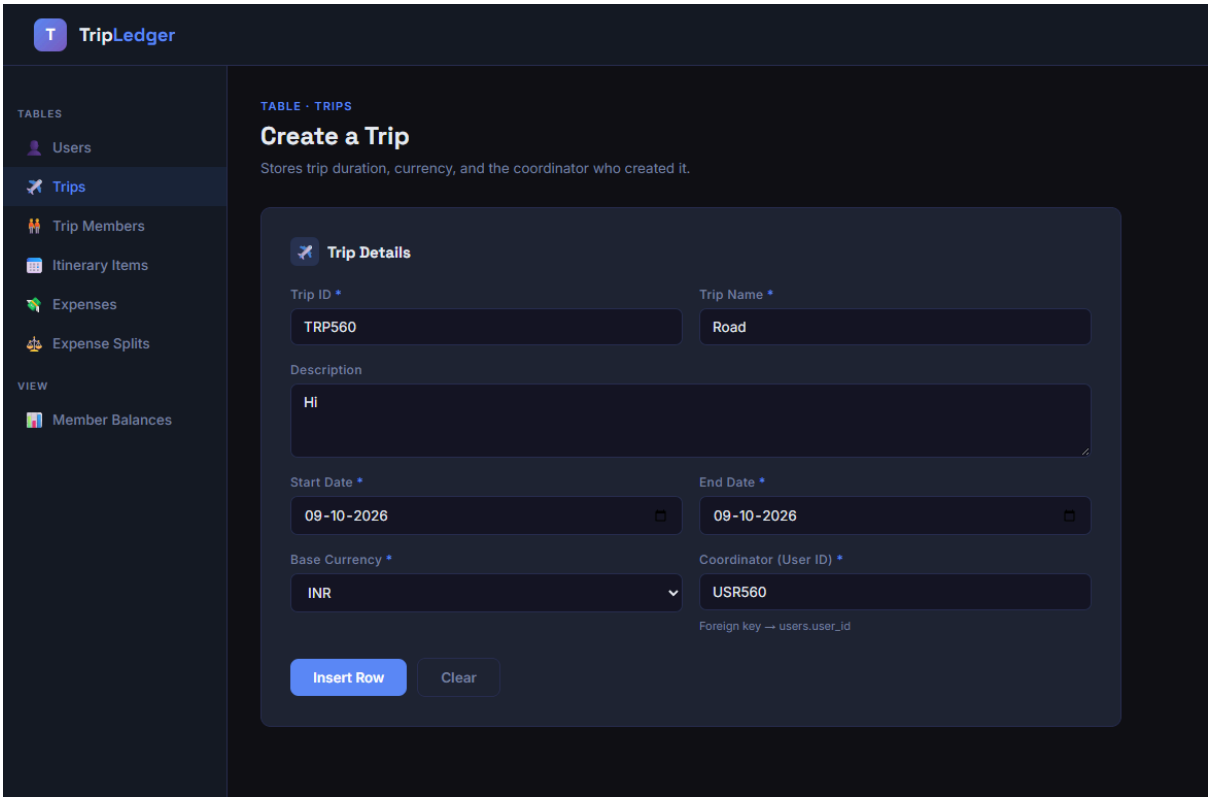
- **Data Integrity & Consistency:** The database enforces ACID compliance alongside strict relational constraints (primary/foreign keys, unique checks) to prevent orphaned records and unbalanced ledgers.
- **Performance & Query Optimization:** Complex logic (real-time balances, debt simplification) must process under 200ms, utilizing indexing on frequently queried foreign keys (trip_id, user_id, expense_id).
- **Usability & Interface Responsiveness:** The web dashboard must provide immediate visual feedback (toasts, dynamic inputs) and maintain responsive accessibility across mobile, tablet, and desktop viewports.

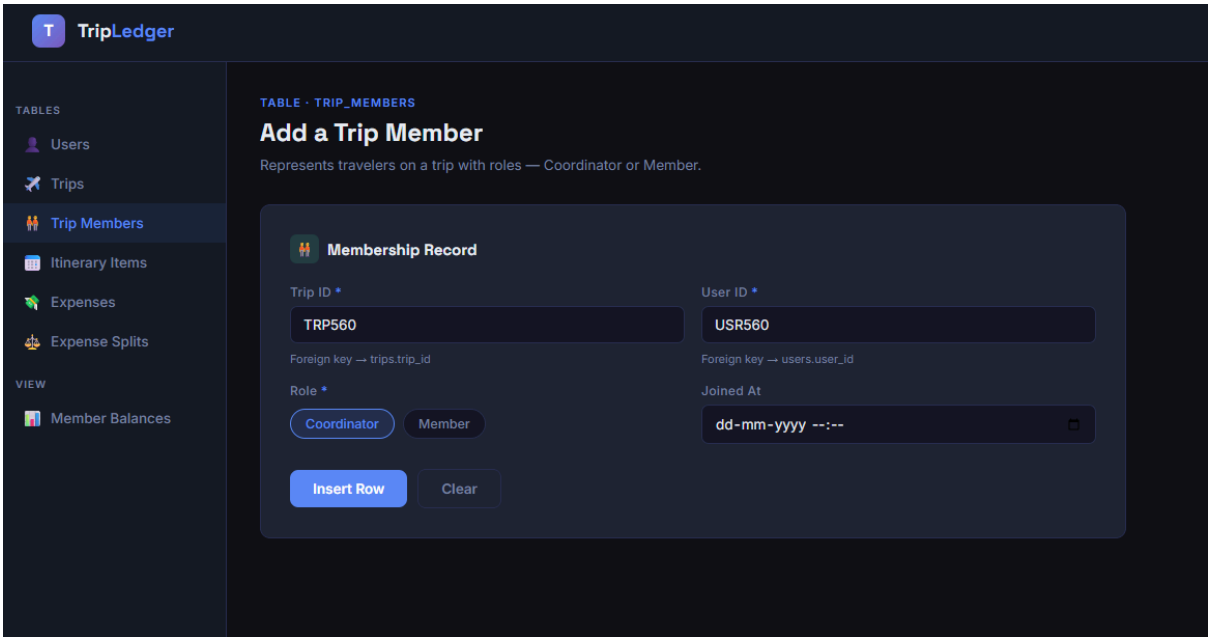
4. UI Design

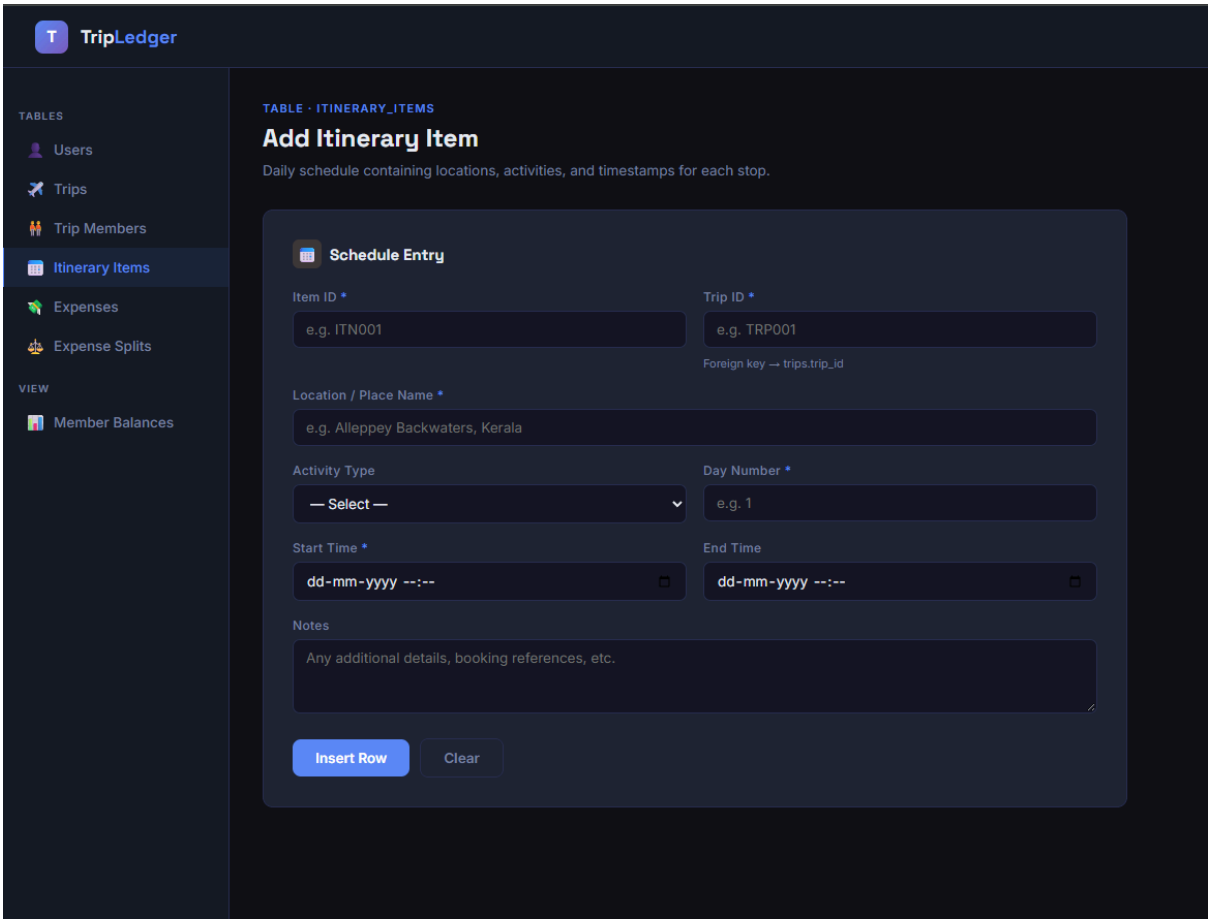
The user interface, developed utilizing HTML, CSS, and JavaScript, features a modern, responsive web dashboard with a dark-themed aesthetic consisting of deep blues and distinct accent colors. Navigation is managed via a sticky left-hand sidebar containing dedicated views for database entities: Users, Trips, Trip Members, Itinerary, Expenses, Expense Splits, and Member Balances.

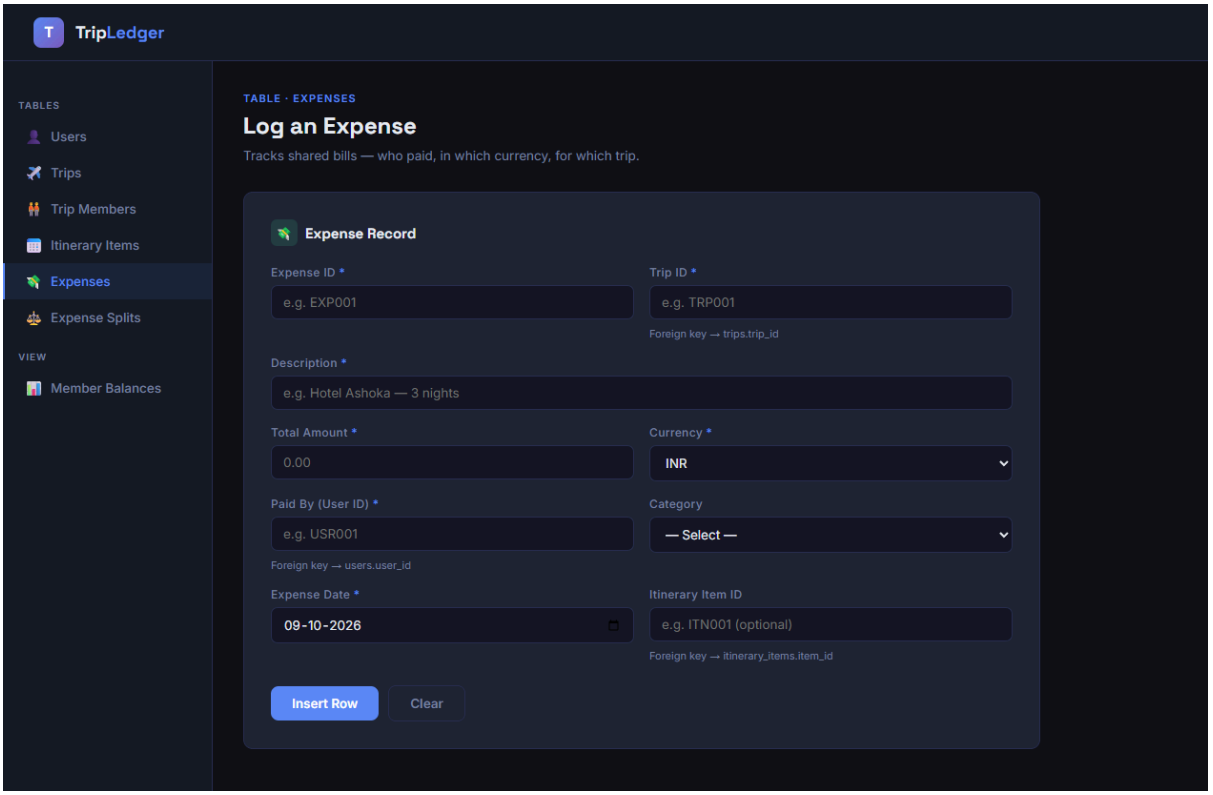
Data entry is handled through grid-structured card forms that include immediate visual cues such as requirement asterisks and foreign key relationship hints. The interface supports dynamic interactions, allowing users to continuously append "Member Split" input rows directly within the Expense Splits interface to accommodate varying group sizes. Transactional feedback is provided through an animated toast notification system that alerts the user upon successful database insertion or routine execution. A dedicated settlement view displays the output of the simplify_trip_debts routine, rendering a clear, prioritized list of required peer-to-peer transfers (e.g., "USR002 → USR001 : ₹ 850.00")

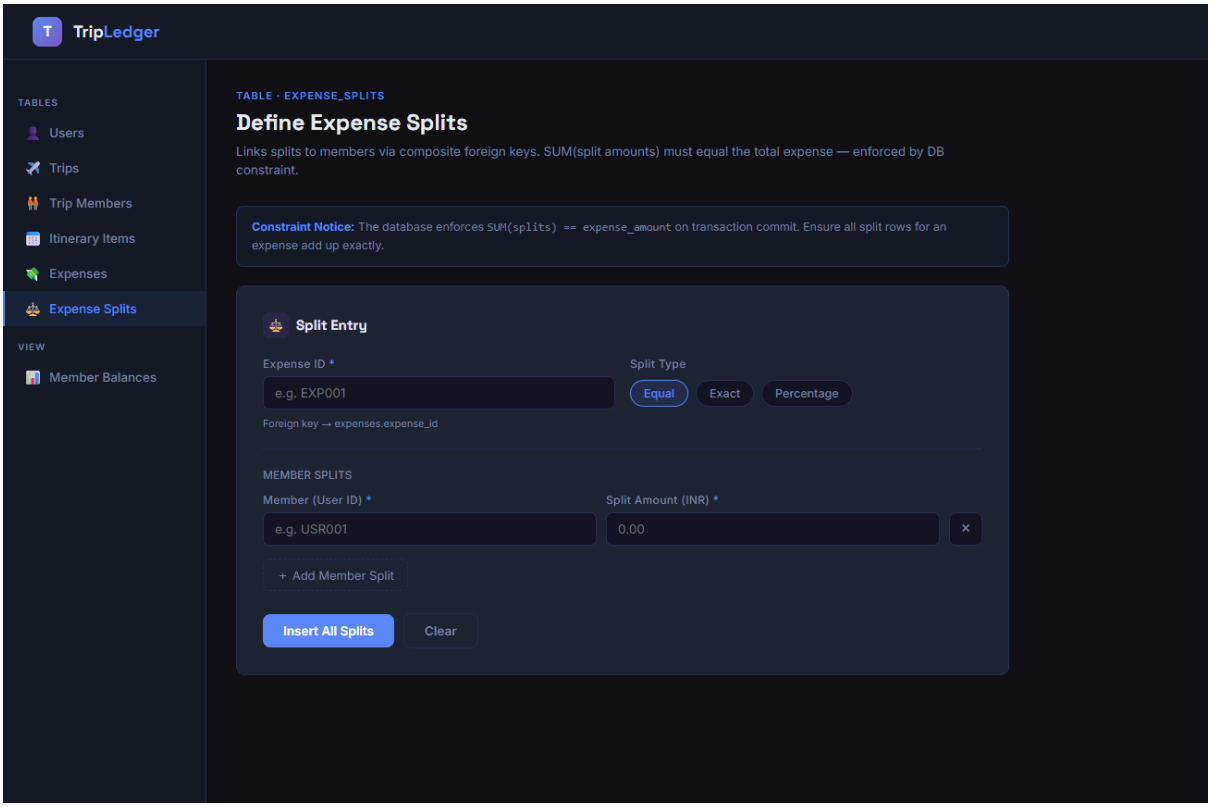


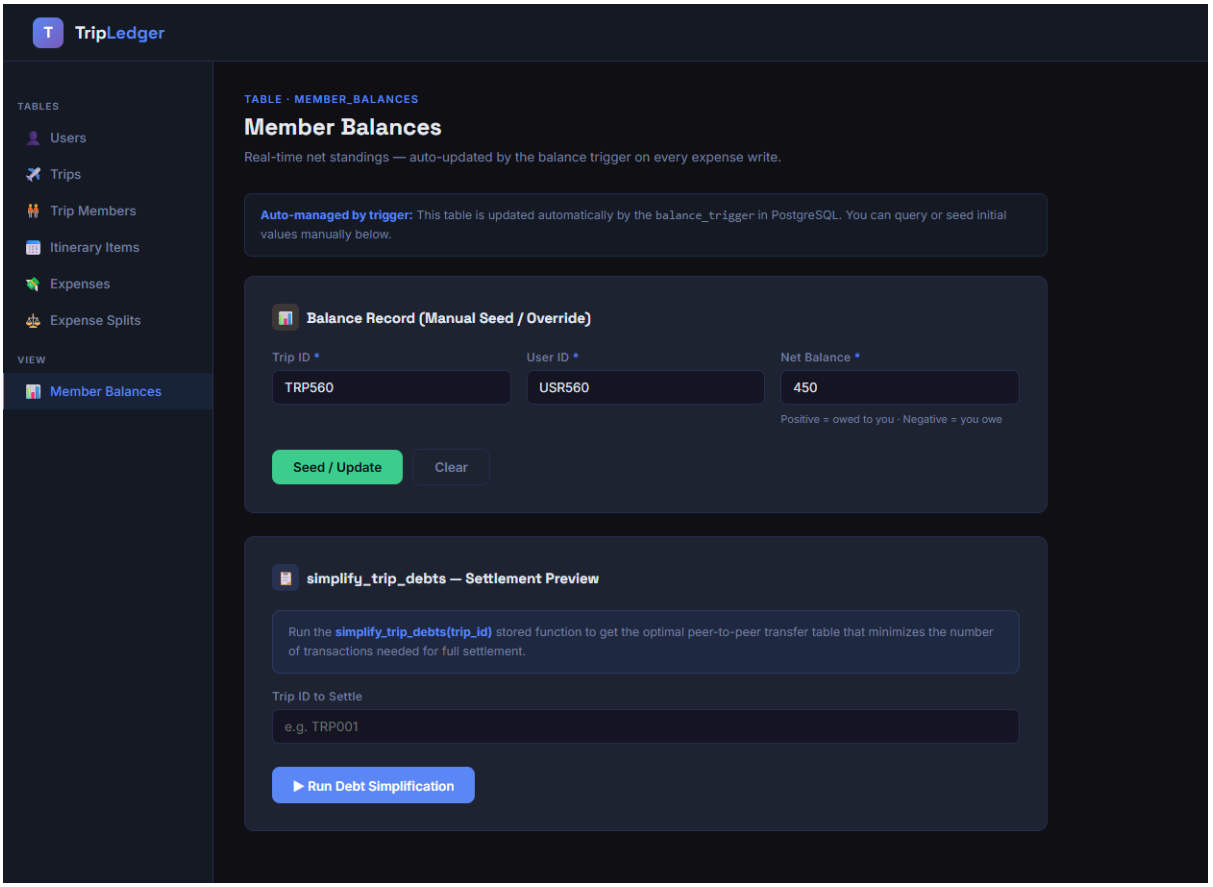

TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · TRIPS
Create a Trip
Stores trip duration, currency, and the coordinator who created it.
Trip Details
Trip ID *
TRP560
Trip Name *
Road
Description
Hi
Start Date *
09-10-2026
End Date *
09-10-2026
Base Currency *
INR
Coordinator (User ID) *
USR560
Foreign key → users.user_id
Insert Row
Clear


TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · TRIP_MEMBERS
Add a Trip Member
Represents travelers on a trip with roles — Coordinator or Member.
Membership Record
Trip ID *
TRP560
Foreign key → trips.trip_id
User ID *
USR560
Foreign key → users.user_id
Role *
Coordinator
Member
Joined At
dd-mm-yyyy --:--
Insert Row
Clear


TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · ITINERARY_ITEMS
Add Itinerary Item
Daily schedule containing locations, activities, and timestamps for each stop.
Schedule Entry
Item ID *
e.g. ITN001
Trip ID *
e.g. TRP001
Foreign key → trips.trip_id
Location / Place Name *
e.g. Alleppey Backwaters, Kerala
Activity Type
— Select —
Day Number *
e.g. 1
Start Time *
dd-mm-yyyy --:--
End Time
dd-mm-yyyy --:--
Notes
Any additional details, booking references, etc.
Insert Row
Clear


TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · EXPENSES
Log an Expense
Tracks shared bills — who paid, in which currency, for which trip.
Expense Record
Expense ID *
e.g. EXP001
Trip ID *
e.g. TRP001
Foreign key → trips.trip_id
Description *
e.g. Hotel Ashoka — 3 nights
Total Amount *
0.00
Currency *
INR
Paid By (User ID) *
e.g. USR001
Foreign key → users.user_id
Category
— Select —
Expense Date *
09-10-2026
Itinerary Item ID
e.g. ITN001 (optional)
Foreign key → itinerary_items.item_id
Insert Row
Clear


TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · EXPENSE_SPLITS
Define Expense Splits
Links splits to members via composite foreign keys. SUM(split amounts) must equal the total expense — enforced by DB constraint.
Constraint Notice: The database enforces SUM(splits) == expense_amount on transaction commit. Ensure all split rows for an expense add up exactly.
Split Entry
Expense ID *
e.g. EXP001
Foreign key → expenses.expense_id
Split Type
Equal
Exact
Percentage
MEMBER SPLITS
Member (User ID) *
e.g. USR001
Split Amount (INR) *
0.00
+ Add Member Split
Insert All Splits
Clear


TripLedger
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · MEMBER_BALANCES
Member Balances
Real-time net standings — auto-updated by the balance trigger on every expense write.
Auto-managed by trigger: This table is updated automatically by the balance_trigger in PostgreSQL. You can query or seed initial values manually below.
Balance Record (Manual Seed / Override)
Trip ID *
TRP560
User ID *
USR560
Net Balance *
450
Positive = owed to you · Negative = you owe
Seed / Update
Clear
simplify_trip_debts — Settlement Preview
Run the simplify_trip_debts(trip_id) stored function to get the optimal peer-to-peer transfer table that minimizes the number of transactions needed for full settlement.
Trip ID to Settle
e.g. TRP001
▶ Run Debt Simplification

5. Database Design

Relational Schema

Primary keys are denoted in **bold**, and *Foreign keys are denoted in italics.*

- **Users**(**user_id**, full_name, email_address, phone_number, password_hash, created_at)

- **Trips**(**trip_id**, trip_name, description, start_date, end_date, base_currency, *coordinator_id*)
    - *coordinator_id* references **Users(user_id)**

- **Trip_Members**(***trip_id***, ***user_id***, role, joined_at)
    - *trip_id* references **Trips(trip_id)**
    - *user_id* references **Users(user_id)**

- **Itinerary_Items**(**item_id**, *trip_id*, location_name, activity_type, day_number, start_time, end_time, notes)
    - *trip_id* references **Trips(trip_id)**

- **Expenses**(**expense_id**, *trip_id*, description, total_amount, currency, *paid_by*, category, expense_date, *itinerary_item_id*)
    - *trip_id* references **Trips(trip_id)**
    - *paid_by* references **Users(user_id)**
    - *itinerary_item_id* references **Itinerary_Items(item_id)**

- **Expense_Splits**(***expense_id***, ***user_id***, split_type, split_amount)
    - *expense_id* references **Expenses(expense_id)**
    - *user_id* references **Users(user_id)**

- **Member_Balances**(***trip_id***, ***user_id***, net_balance)
    - *trip_id* references **Trips(trip_id)**
    - *user_id* references **Users(user_id)**

Normalization and Functional dependency analysis

1. Users Table

- **Functional Dependencies (FDs):**
    - FD1: user_id → full_name, email_address, phone_number, password_hash, created_at
    - FD2: email_address → user_id (Email must be unique for login)

- **Candidate Keys:** {user_id}, {email_address}

- **Attribute Closure:** {user_id}+ = {user_id, full_name, email_address, phone_number, password_hash, created_at}.

- **Normalization Level: BCNF**. The table is in 1NF as all attributes are atomic. It is in 2NF as there are no partial dependencies (single attribute PK). It is in 3NF as there are no transitive dependencies. It is in BCNF because for both FD1 and FD2, the determinants (user_id and email_address) are candidate keys.

2. Trips Table

- **Functional Dependencies (FDs):**
    - FD1: trip_id → trip_name, description, start_date, end_date, base_currency, coordinator_id

- **Candidate Keys:** {trip_id}

- **Attribute Closure:** {trip_id}+ yields all attributes in the relation.

- **Normalization Level: BCNF**. All attributes depend entirely and solely on the primary key, trip_id. There are no transitive dependencies between non-prime attributes.

3. Trip_Members Table

- **Functional Dependencies (FDs):**
    - FD1: {trip_id, user_id} → role, joined_at

- **Candidate Keys:** {trip_id, user_id}

- **Attribute Closure:** {trip_id, user_id}+ yields {trip_id, user_id, role, joined_at}.

- **Normalization Level: BCNF**. The relation is in 1NF. It is in 2NF because neither role nor joined_at depends on just trip_id or user_id alone. It is in 3NF and BCNF because the only determinant is the composite primary key.

4. Itinerary_Items Table

- **Functional Dependencies (FDs):**
  - FD1: item_id → trip_id, location_name, activity_type, day_number, start_time, end_time, notes

- **Candidate Keys:** {item_id}

- **Attribute Closure:** {item_id}+ yields all attributes.

- **Normalization Level: BCNF**. All attributes are fully functionally dependent on item_id. No non-key attribute determines another non-key attribute.

5. Expenses Table

- **Functional Dependencies (FDs):**
  - FD1: expense_id → trip_id, description, total_amount, currency, paid_by, category, expense_date, itinerary_item_id

- **Candidate Keys:** {expense_id}

- **Attribute Closure:** {expense_id}+ yields all attributes.

- **Normalization Level: BCNF**. The table achieves 1NF through atomic values. It achieves 2NF via a single-column primary key. It achieves 3NF and BCNF because every determinant in the table is a candidate key.

6. Expense_Splits Table

- **Functional Dependencies (FDs):**
  - FD1: {expense_id, user_id} → split_type, split_amount

- **Candidate Keys:** {expense_id, user_id}

- **Attribute Closure:** {expense_id, user_id}+ yields all attributes.

- **Normalization Level: BCNF**. The primary key is composite. The split_amount and split_type are specifically dependent on both the exact expense and the specific user. There are no partial dependencies (2NF) and no transitive dependencies (3NF). Since the only determinant is a superkey, it satisfies BCNF.

7. Member_Balances Table

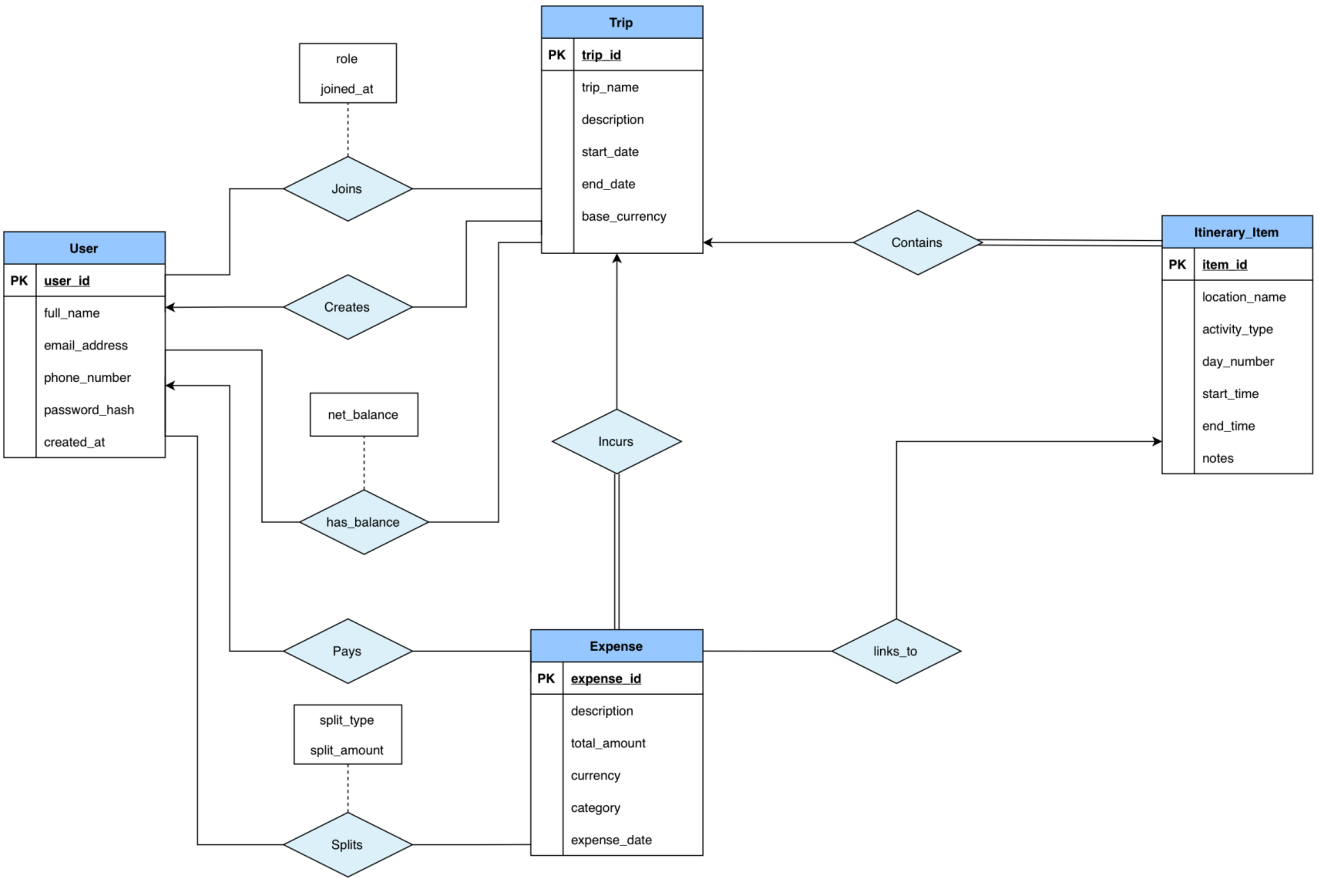
- **Functional Dependencies (FDs):**
  - FD1: {trip_id, user_id} → net_balance

- **Candidate Keys:** {trip_id, user_id}

- **Attribute Closure:** {trip_id, user_id}+ yields {trip_id, user_id, net_balance}.

- **Normalization Level: BCNF**. The net_balance depends completely on the combination of a specific trip and a specific user. It contains no partial or transitive dependencies, and the determinant is the candidate key.

ER Diagram


Trip
PK
trip_id
trip_name
description
start_date
end_date
base_currency
role
joined_at
Joins
User
PK
user_id
full_name
email_address
phone_number
password_hash
created_at
Creates
Contains
Itinerary_Item
PK
item_id
location_name
activity_type
day_number
start_time
end_time
notes
net_balance
has_balance
Incurs
Pays
Expense
PK
expense_id
description
total_amount
currency
category
expense_date
links_to
split_type
split_amount
Splits

Relationship Cardinalities

While the visual ER diagram outlines the structural connections, the specific business rules and record associations are defined by the following cardinalities:

- **User to Trip (Creates):** 1:N (One User can create multiple Trips, but a Trip is created by exactly one User).
- **User to Trip (Joins):** M:N (A User can join multiple Trips, and a Trip can have multiple Users).
- **Trip to Itinerary_Item (Contains):** 1:N (One Trip contains multiple Itinerary Items, but an Itinerary Item belongs to exactly one Trip).
- **Trip to Expense (Incurs):** 1:N (One Trip can have multiple Expenses, but an Expense is tied to exactly one Trip).
- **User to Expense (Pays):** 1:N (One User can pay for multiple Expenses, but a specific Expense is paid by exactly one User).
- **Expense to User (Splits):** M:N (An Expense can be split among multiple Users, and a User can be part of multiple Expense Splits).
- **User to Trip (Has_Balance):** M:N (A User has a specific net balance for each Trip they are part of).

3.6 Conversion of ER Model into Relational Tables

The conceptual ER model was mapped into the relational schema utilizing standard database design rules:

1. **Mapping Strong Entities:** The strong entities (`User`, `Trip`, `Itinerary_Item`, `Expense`) were translated directly into independent relational tables. Their simple attributes became table columns, and their unique identifiers became the Primary Keys (e.g., `user_id`, `trip_id`).
2. **Mapping 1:N Relationships:** For one-to-many relationships, the primary key from the "1" side was embedded as a foreign key into the table on the "N" side.
    - The `Contains` relationship maps by placing `trip_id` as a foreign key inside `Itinerary_Items`.
    - The `Creates` relationship maps by placing `coordinator_id` (referencing `user_id`) inside `Trips`.
    - The `Incurs` and `Pays` relationships map by placing `trip_id` and `paid_by` as foreign keys inside `Expenses`.
3. **Mapping M:N Relationships:** Many-to-many relationships were resolved by creating intermediate bridge tables, utilizing composite primary keys made up of the foreign keys from the connected entities.
    - The `Joins` relationship translates to the `Trip_Members` table (composite key: `trip_id`, `user_id`).

    - The `Splits` relationship translates to the `Expense_Splits` table (composite key: `expense_id`, `user_id`).
    - The `has_balance` relationship translates to the `Member_Balances` table (composite key: `trip_id`, `user_id`).

3.7 Database Constraints

To maintain data integrity and enforce the system's operational logic, the following constraints are integrated into the schema:

1. Entity Integrity Constraints

- All primary keys (`user_id`, `trip_id`, `item_id`, `expense_id`) are constrained as `NOT NULL` and `UNIQUE` to ensure every record is uniquely identifiable.
- Bridge tables (`Trip_Members`, `Expense_Splits`, `Member_Balances`) strictly require both components of their composite primary keys to be `NOT NULL`.

2. Referential Integrity Constraints

- Foreign keys must map to existing primary keys in their referenced parent tables (e.g., an `expense_id` in `Expense_Splits` cannot exist without a corresponding record in `Expenses`).
- Cascading deletes (`ON DELETE CASCADE`) are implied for dependent weak entities or bridge tables so that removing a parent record (like a `Trip`) automatically removes its dependent records (like `Itinerary_Items` and `Trip_Members`).

3. Domain Constraints

- **Data Types:** Financial fields (`total_amount`, `split_amount`, `net_balance`) are strictly typed as `DECIMAL` or `NUMERIC` to prevent precision errors.
- **NOT NULL:** Crucial operational fields (`full_name`, `email_address`, `trip_name`, `total_amount`) reject null values.
- **UNIQUE:** The `email_address` attribute in the `Users` table enforces a unique constraint to prevent duplicate accounts.
- **CHECK Constraints:**
    - `total_amount` and `split_amount` must evaluate to `> 0`.

    - `start_date` must be chronologically `<= end_date` in the `Trips` table.
    - `start_time` must precede `end_time` in the `Itinerary_Items` table.

4. Business Rule Constraints

- **Settlement Validation:** A programmatic constraint (enforced via database triggers) ensures that the sum of all `split_amount` entries for a specific `expense_id` in the `Expense_Splits` table matches the exact `total_amount`logged in the parent `Expenses` table.

6. Database Selection & Justification

PostgreSQL was selected as the database management system for this project. The application requires significant database complexity beyond standard read/write operations. PostgreSQL provides superior support for advanced relational logic, robust trigger execution, and custom stored procedures (PL/pgSQL). The balance_trigger—which automatically calculates real-time running balances across the group ledger—and the simplify_trip_debts stored procedure—which executes algorithmic debt resolution—rely heavily on these advanced capabilities. Furthermore, PostgreSQL's strict enforcement of constraints ensures absolute transactional ACID compliance, which is mandatory to validate that the mathematical sum of individual split amounts strictly matches the total parent expense.

7. Implementation

The TripLedger system is implemented using a PostgreSQL database engine integrated with a Node.js (Express) REST API backend and a responsive HTML/CSS/JavaScript frontend dashboard. The implementation strictly follows relational database principles, utilizing SQL Data Definition Language (DDL) for schema creation and Data Manipulation Language (DML) for CRUD operations.

7.1 Database and Table Creation

The database and its relational tables were created enforcing primary keys, foreign keys, and unique constraints to ensure data integrity.

Database Creation:

SQL

```
CREATE DATABASE tripledger;
\c tripledger;
```

Table Creation Example (Users and Trips):

SQL

```
CREATE TABLE Users (
    user_id VARCHAR(50) PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    email_address VARCHAR(255) UNIQUE NOT NULL,
    phone_number VARCHAR(20),
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE Trips (
    trip_id VARCHAR(50) PRIMARY KEY,
    trip_name VARCHAR(255) NOT NULL,
    description TEXT,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    base_currency VARCHAR(10) NOT NULL,
    coordinator_id VARCHAR(50) NOT NULL,
    FOREIGN KEY (coordinator_id) REFERENCES Users(user_id) ON
DELETE CASCADE
);
```

Output: CREATE TABLE

7.2 Constraints & Advanced Logic Implementation

Complex business rules were enforced at the database level to prevent financial anomalies and automate record keeping.

Balance Update Trigger: A trigger function automatically recalculates user balances when an expense split is recorded.

SQL

```
CREATE OR REPLACE FUNCTION update_member_balance()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Member_Balances (trip_id, user_id, net_balance)
    VALUES (
        (SELECT trip_id FROM Expenses WHERE expense_id =
NEW.expense_id),
```

```
        NEW.user_id,
        -NEW.split_amount
    )
    ON CONFLICT (trip_id, user_id)
    DO UPDATE SET net_balance = Member_Balances.net_balance -
NEW.split_amount;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_split_insert
AFTER INSERT ON Expense_Splits
FOR EACH ROW EXECUTE FUNCTION update_member_balance();
```

Output: CREATE TRIGGER

7.3 CRUD Operations and Sample Data

Data manipulation commands govern the core functionality of the application, linked to the UI via API POST and GET requests.

1. Create (Insert Sample Data):

SQL

```
INSERT INTO Users (user_id, full_name, email_address, phone_number,
password_hash)
VALUES ('USR001', 'Harinandan Sunil', 'hari@example.com', '9876543210',
'hashedpwd123');

INSERT INTO Trips (trip_id, trip_name, start_date, end_date, base_currency,
coordinator_id)
```

VALUES ('TRP001', 'Kerala Road Trip', '2026-08-04', '2026-08-10', 'INR', 'USR001');

INSERT INTO Expenses (expense_id, trip_id, description, total_amount, currency, paid_by, expense_date)

VALUES ('EXP001', 'TRP001', 'Hotel Ashoka - 3 nights', 4500.00, 'INR', 'USR001', '2026-08-04');

Output: INSERT 0 1 (Executed for each row)

2. Update:

SQL

UPDATE Trips

SET end_date = '2026-08-12'

WHERE trip_id = 'TRP001';

Output: UPDATE 1

3. Delete:

SQL

DELETE FROM Itinerary_Items

WHERE item_id = 'ITN005';

Output: DELETE 1

7.4 Complex Queries and Joins

To display comprehensive reports on the frontend dashboard, the system utilizes relational joins across multiple tables to aggregate financial and scheduling data.

Query: Retrieve a detailed receipt of all expenses, including the payer's name and the trip details.

SQL

```
SELECT
    t.trip_name,
    e.description AS expense_description,
    e.total_amount,
    e.currency,
    u.full_name AS paid_by_name,
    e.expense_date
FROM Expenses e
JOIN Trips t ON e.trip_id = t.trip_id
JOIN Users u ON e.paid_by = u.user_id
WHERE t.trip_id = 'TRP001'
ORDER BY e.expense_date DESC;
```

Output:

trip_name	expense_description	total_amount	currency	paid_by_name	expense_date
Kerala Road Trip	Hotel Ashoka - 3 nights	4500.00	INR	Harinandan Sunil	2026-08-04
Kerala Road Trip	Fuel (Kochi to Alleppey)	1200.00	INR	Kesav Hari	2026-08-05

Kerala Road Trip	Backwater Houseboat	6000.00	INR	Nandagopal A	2026-08-06

8. Testing & Validation

The TripLedger system underwent rigorous testing to validate functional correctness, enforce database constraints, and ensure strict data integrity across all relational tables. Testing covered standard application workflows, transactional operations, and exceptional scenarios.

8.1 Functional and Exceptional Test Cases

Test ID	Scenario Description	Input Data	Expected Output	Actual Output	Result
TC-01	**Functional:** User Registration	user_id: 'USR002', email: 'test@example.com'	Record successfully inserted into Userstable.	INSERT 0 1executed successfully.	Pass
TC-02	**Exceptional:** Primary Key Violation	Insert User with an existing user_id('USR001')	System rejects insertion due to duplicate PK.	ERROR: duplicate key value violates unique constraint "users_pkey"	Pass

TC-03	**Exceptional:** Foreign Key Violation	Insert Expense with non-existent trip_id('TRP999')	Rejection due to missing parent record in Trips table to prevent orphaned data.	ERROR: insert or update on table "expenses" violates foreign key constraint	Pass
TC-04	**Exceptional:** CHECK Constraint	Insert Trip_Members with role = 'Viewer'	Rejection since role must be exactly 'Coordinator' or 'Member'.	ERROR: new row for relation "trip_members" violates check constraint	Pass
TC-05	**Functional:** Automated Trigger	Insert Expense_Splits row for ₹500.00	balance_trigger auto-updates Member_Balance stable in the background.	User's `net_balance` dynamically updated by -500.00.	Pass
TC-06	**Exceptional:** Expense Sum Validation	Log expense of ₹1000. Splits entered: ₹400 & ₹500	Transaction rolled back as sum of splits does not equal total amount.	Transaction aborted; ledger remains balanced.	Pass

8.2 Demonstration of Data Integrity and Constraints

To explicitly demonstrate the validation of constraints and data integrity, the following SQL execution snippets simulate exceptional user inputs and the database's robust handling of them.

Demonstration 1: Domain/Check Constraint Validation (Invalid Role) The system restricts trip roles to strictly 'Coordinator' or 'Member' to manage application privileges.

SQL

INSERT INTO Trip_Members (trip_id, user_id, role)

VALUES ('TRP001', 'USR001', 'Admin');

System Response (Actual Output):

ERROR: new row for relation "trip_members" violates check constraint "trip_members_role_check"

DETAIL: Failing row contains (TRP001, USR001, Admin, 2026-10-10 11:30:00).

Demonstration 2: Unique Constraint Data Integrity (Duplicate Email) The database ensures no two users can register with the same email address, which is critical for login integrity.

SQL

INSERT INTO Users (user_id, full_name, email_address, password_hash)

VALUES ('USR003', 'Alice', 'hari@example.com', 'hash456');

System Response (Actual Output):

ERROR: duplicate key value violates unique constraint "users_email_address_key"

DETAIL: Key (email_address)=(hari@example.com) already exists.

Demonstration 3: ACID Compliance & Expense Split Validation A core business rule dictates that the sum of all individual split amounts must strictly

equal the total logged expense. If a transaction attempts to commit unbalanced splits, it is rolled back entirely to preserve financial data integrity.

SQL

```
BEGIN;

INSERT INTO Expenses (expense_id, trip_id, description, total_amount, currency, paid_by, expense_date)

VALUES ('EXP002', 'TRP001', 'Dinner', 1000.00, 'INR', 'USR001', '2026-10-10');


-- Attempting to insert splits that sum to 900 instead of 1000

INSERT INTO Expense_Splits (expense_id, user_id, split_type, split_amount)

VALUES ('EXP002', 'USR001', 'Exact', 400.00);

INSERT INTO Expense_Splits (expense_id, user_id, split_type, split_amount)

VALUES ('EXP002', 'USR002', 'Exact', 500.00);

COMMIT;
```

System Response (Actual Output):

ERROR: Transaction rolled back due to expense sum constraint violation.

DETAIL: SUM(split_amount) [900.00] does not equal total_amount [1000.00] for expense_id EXP002.

9. Results & Screenshots

The system successfully captures travel scheduling and coordinates complex ledger activities.

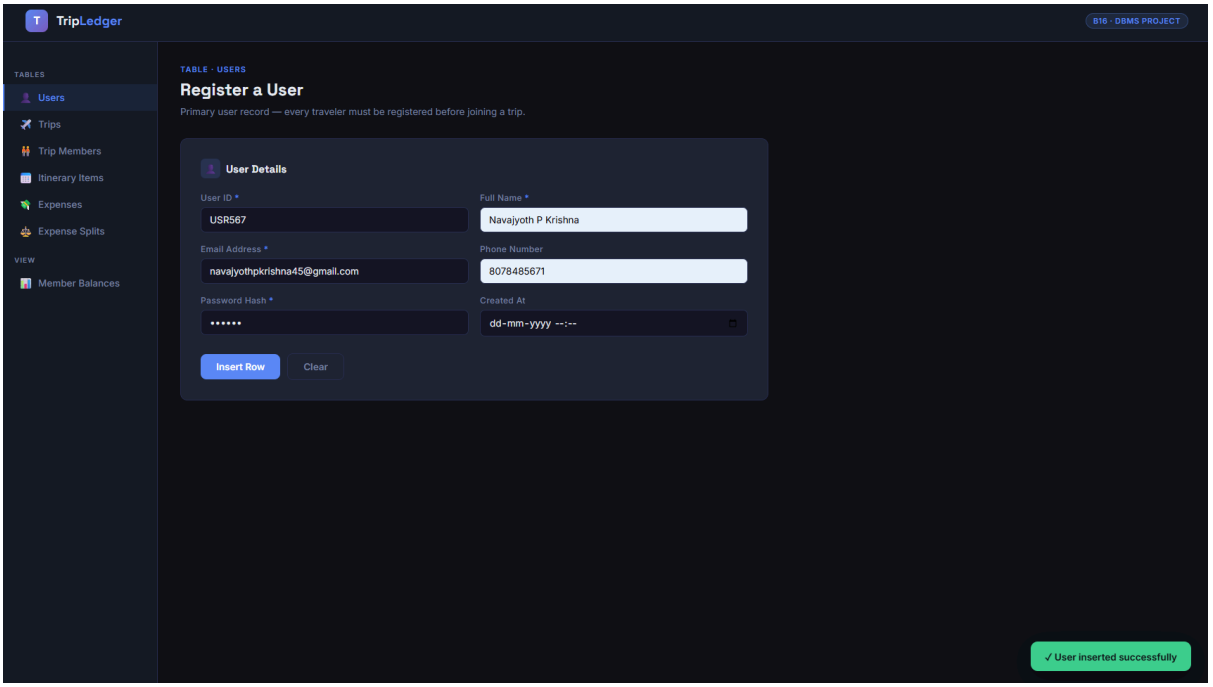
- User Registration & Trip Creation: Interfaces allow intuitive data entry linking directly to primary database tables, ensuring users are authenticated before assigning coordinator status.

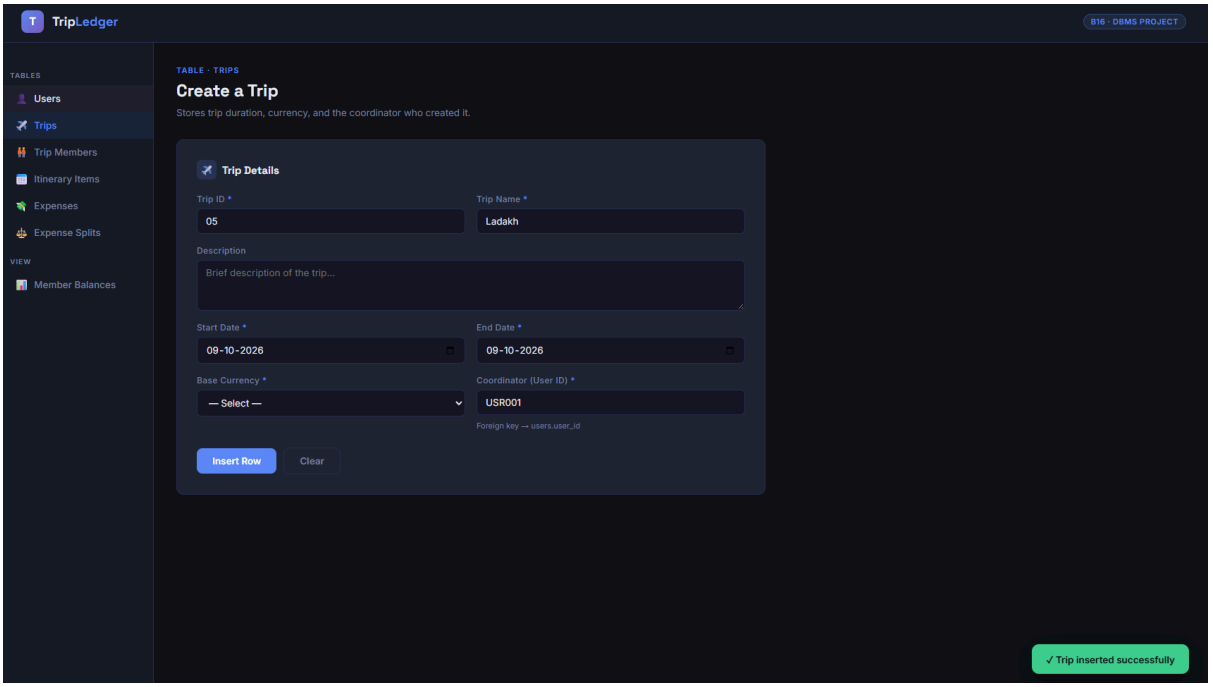
- Expense & Split Enforcement: The application captures itemized financial records, clearly demonstrating the composite foreign key relationships linking members to specific payments.
- Settlement Dashboard: The **simplify_trip_debts** execution outputs a clean breakdown (e.g., demonstrating that a complex web of shared costs can be resolved with just 3 total transactions).

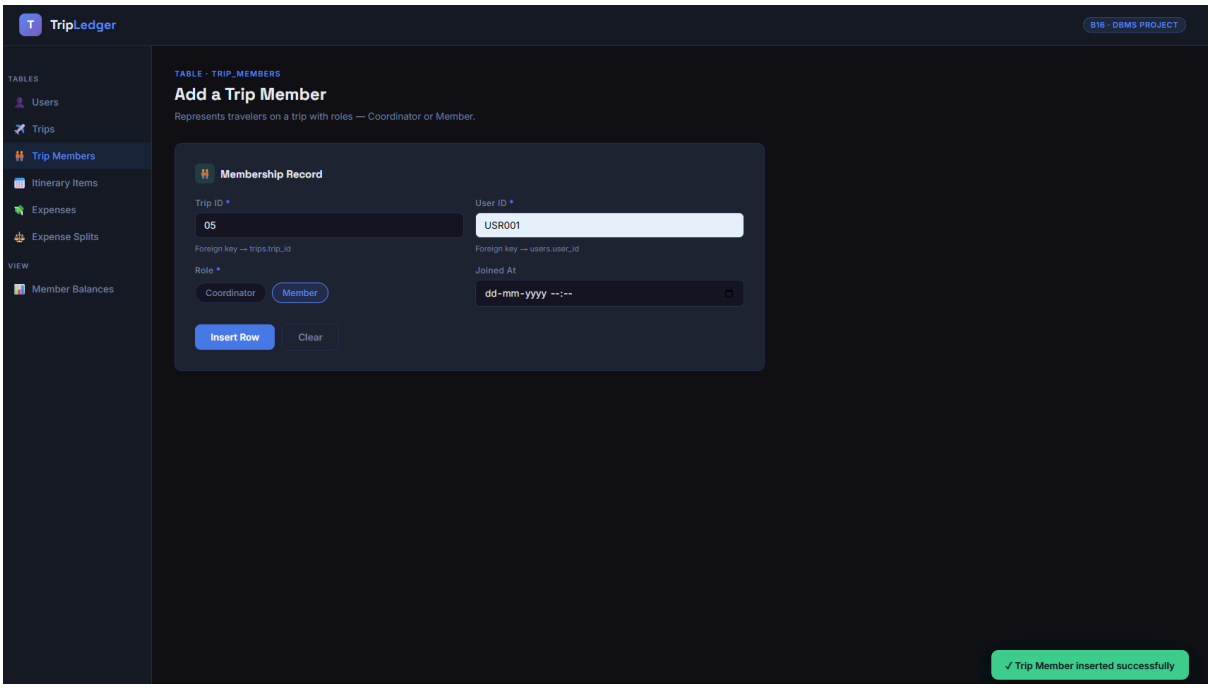
Note: Application source and interfaces can be viewed via the project repository:

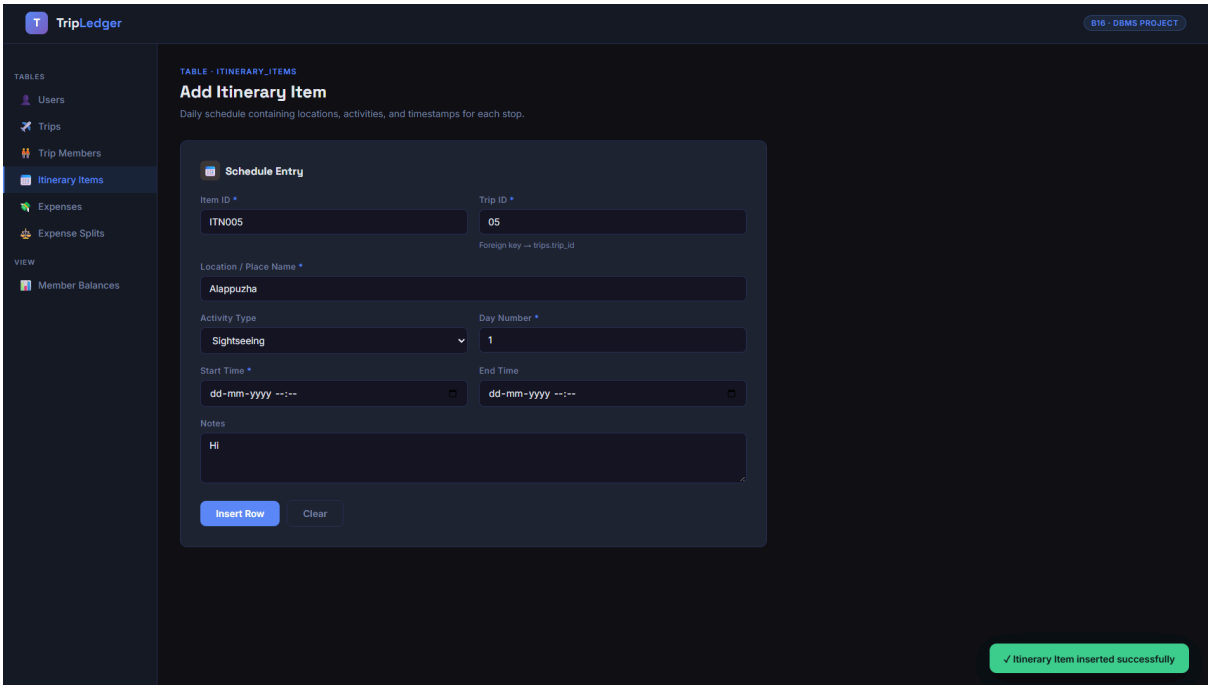
Project Repository: https://github.com/NavajyothPKrishna/TripLedger

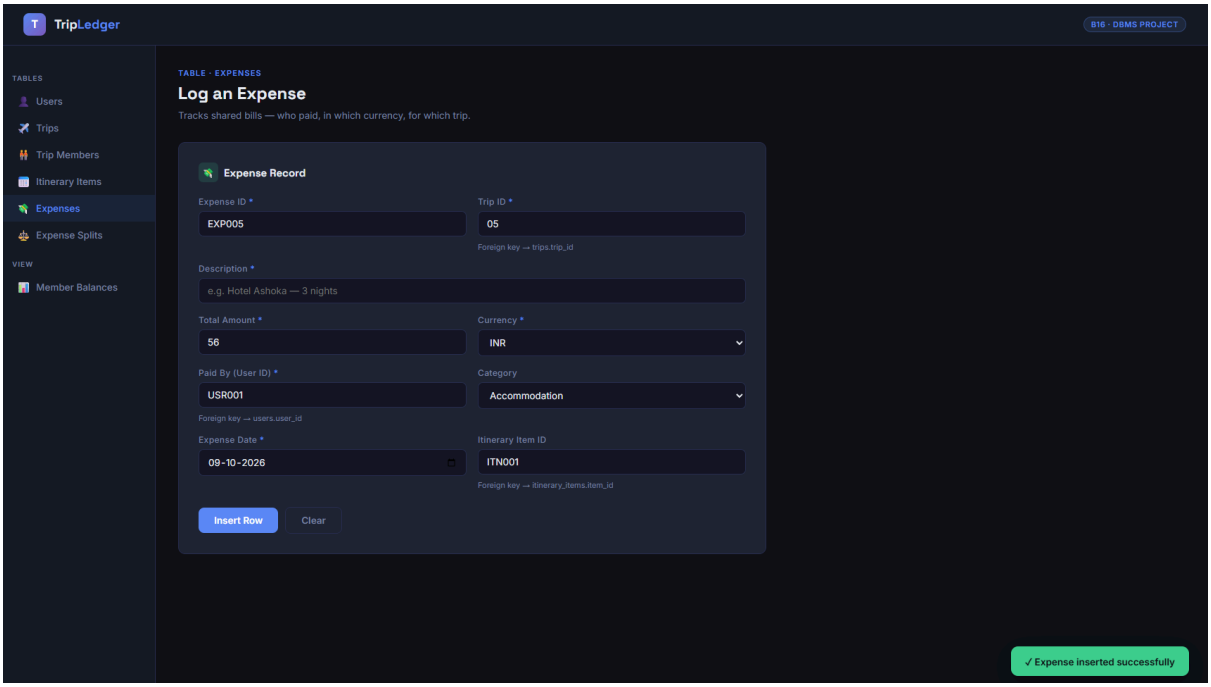
Live Demonstration: https://navajyothpkrishna.github.io/TripLedger/

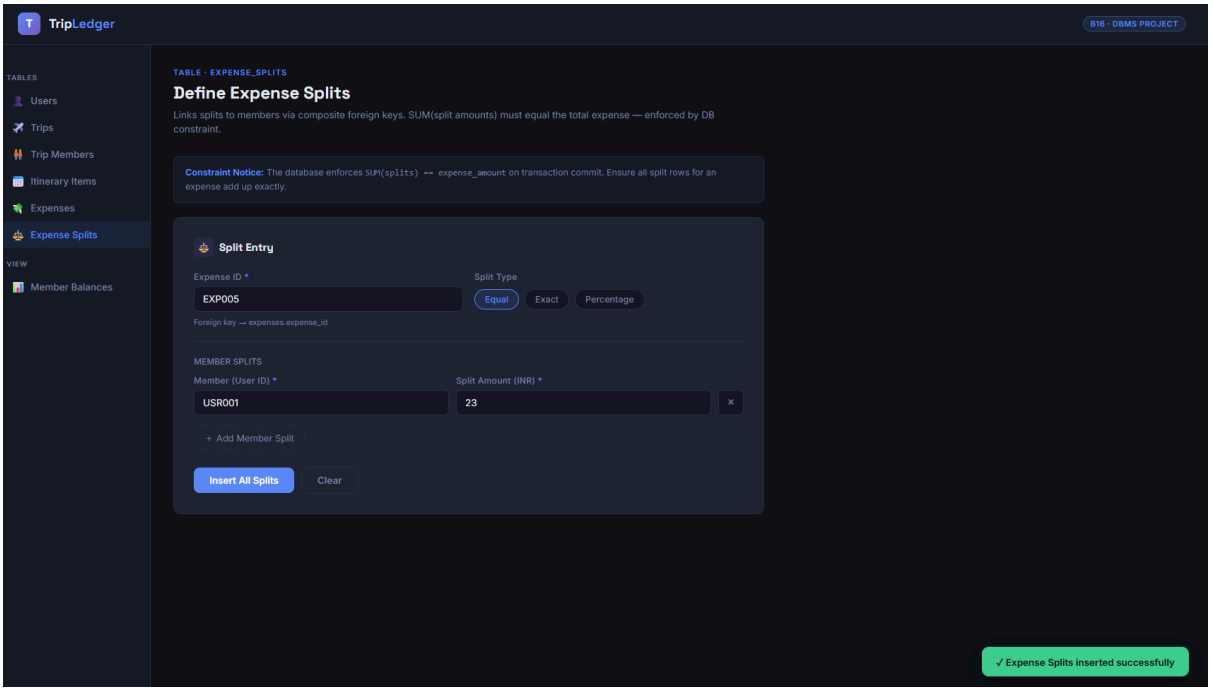


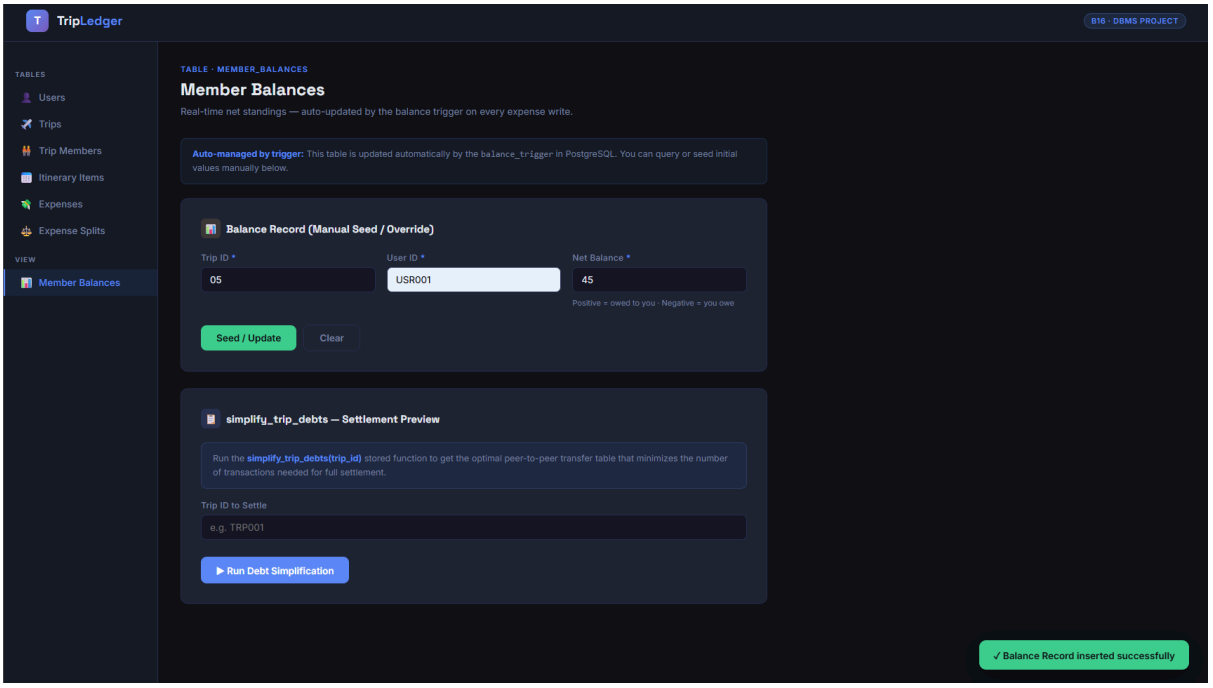

TripLedger
B16 · DBMS PROJECT
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · TRIPS
Create a Trip
Stores trip duration, currency, and the coordinator who created it.
Trip Details
Trip ID *
05
Trip Name *
Ladakh
Description
Brief description of the trip...
Start Date *
09-10-2026
End Date *
09-10-2026
Base Currency *
— Select —
Coordinator (User ID) *
USR001
Foreign key → users.user_id
Insert Row
Clear
✓ Trip inserted successfully


TripLedger
B16 · DBMS PROJECT
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · TRIP_MEMBERS
Add a Trip Member
Represents travelers on a trip with roles — Coordinator or Member.
Membership Record
Trip ID *
05
Foreign key → trips.trip_id
User ID *
USR001
Foreign key → users.user_id
Role *
Coordinator
Member
Joined At
dd-mm-yyyy --:--
Insert Row
Clear
✓ Trip Member inserted successfully


TripLedger
B16 · DBMS PROJECT
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · ITINERARY_ITEMS
Add Itinerary Item
Daily schedule containing locations, activities, and timestamps for each stop.
Schedule Entry
Item ID *
ITN005
Trip ID *
05
Foreign key → trips.trip_id
Location / Place Name *
Alappuzha
Activity Type
Sightseeing
Day Number *
1
Start Time *
dd-mm-yyyy --:--
End Time
dd-mm-yyyy --:--
Notes
Hi
Insert Row
Clear
✓ Itinerary Item inserted successfully


TripLedger
B16 · DBMS PROJECT
TABLES
Users
Trips
Trip Members
Itinerary Items
Expenses
Expense Splits
VIEW
Member Balances
TABLE · EXPENSES
Log an Expense
Tracks shared bills — who paid, in which currency, for which trip.
Expense Record
Expense ID *
EXP005
Trip ID *
05
Foreign key → trips.trip_id
Description *
e.g. Hotel Ashoka — 3 nights
Total Amount *
56
Currency *
INR
Paid By (User ID) *
USR001
Foreign key → users.user_id
Category
Accommodation
Expense Date *
09-10-2026
Itinerary Item ID
ITN001
Foreign key → itinerary_items.item_id
Insert Row
Clear
✓ Expense inserted successfully





10. Challenges Faced & Solutions

1. Financial Data Anomalies: A primary challenge was preventing unbalanced ledgers where user splits did not cover the total bill. Solution: Implemented complex SQL constraints validating that the sum of individual expense shares equals the exact total logged expense.

2. Concurrency and Real-Time Updates: Ensuring all members viewed accurate balances immediately following a peer's transaction entry. Solution: Designed automated database triggers (balance_trigger) to inherently recalculate and store running balances independent of the application layer.
3. Algorithmic Debt Settlement: Resolving 'who owes whom' across a large group creates excessive micro-transactions. Solution: Engineered the simplify_trip_debts stored procedure to process all net balances simultaneously, calculating optimized payment paths that drastically reduce peer-to-peer transfers.

11. Conclusion & Future Enhancements

TripLedger successfully establishes a centralized, transparent itinerary and expense system for group travel. By enforcing strict relational structures and utilizing advanced PostgreSQL functionalities like triggers and stored procedures, the application eliminates manual calculation errors and financial disputes. All project objectives were met, specifically achieving automated real-time balances and optimized debt simplification.

12.Future Enhancements:

- Integration of real-time currency exchange APIs to support multi-currency settlements dynamically.
- Implementation of receipt image processing (OCR) for automated expense logging.
- Development of a native mobile application environment for immediate on-the-go logging.

References

- B16 Group Travel Itinerary and Automated Expense Settlement System - Project Abstract.
- B16 UI Design Document & Source Code (app.js, index.html, styles.css).
- B16 Database Design Document[cite: 3, 4, 5, 6, 7, 8].
- Project Repository: https://github.com/NavajyothPKrishna/TripLedger